

What you're describing is a classic issue in mobile development called **"Safe Area" management**.

Because modern iPhones have "notches" (the Dynamic Island or top cutout) and "Home Indicators" (the thin line at the bottom), Capacitor tries to fill the entire screen ("edge-to-edge"). If your code doesn't account for these areas, your buttons will hide behind the clock, the battery icon, or the bottom swipe bar.

Here are the three solutions you can apply to fix this without needing a full code rewrite.

1. The "Viewport Fit" Fix (The Most Common Fix)

Your app needs to tell the iPhone that it is aware of the notch. This is done in your main HTML file.

1. Open `src/index.html`.
 2. Find the `<meta name="viewport" ...>` tag.
 3. Add `viewport-fit=cover` to the content attribute.
 - **Before:** `content="width=device-width, initial-scale=1.0"`
 - **After:** `content="width=device-width, initial-scale=1.0, viewport-fit=cover"`
-

2. The CSS "Safe Area" Variables

Once you add `viewport-fit=cover`, iOS provides special "Safe Area" variables that tell your app exactly how many pixels to "push" the content away from the edges. You can apply these globally in your main CSS file (e.g., `src/App.css` or `index.css`).

Add this to your global CSS:

```
CSS
body {
  /* Pushes the top of the app down so it doesn't hit the clock/notch */
  padding-top: env(safe-area-inset-top);

  /* Pushes the bottom of the app up so buttons aren't under the home bar */
  padding-bottom: env(safe-area-inset-bottom);

  /* Ensures side-to-side content doesn't hit the screen curves */
}
```

```
padding-left: env(safe-area-inset-left);
padding-right: env(safe-area-inset-right);
}
```

3. Use "Dynamic Viewport Units" (The Modern Fix)

If you are using `height: 100vh` in your code, it often calculates the height incorrectly on mobile because it ignores the address bar or system menus.

- **The Fix:** Change `100vh` to `100dvh` (Dynamic Viewport Height).
 - `100dvh` automatically shrinks and grows based on whether system menus are visible, preventing your bottom buttons from getting pushed off-screen.
-

4. Capacitor Config Adjustment (The "Quick & Dirty")

If you don't want to mess with CSS, you can tell Capacitor to manually "shrink" the window so it never touches the status bar.

1. Open `capacitor.config.ts`.
2. Add or update the `ios` object:

```
TypeScript
const config: CapacitorConfig = {
  // ... other settings
  ios: {
    contentInset: 'always' // This forces the app to stay within the safe bounds
  }
};
```

Note: This might result in a "black bar" at the top/bottom, which doesn't look as "native" as the CSS fix, but it's the fastest way to make buttons accessible.

Which one should you try first?

I recommend **Solution 1 and 2 together**. They are the professional standard for iOS apps. By adding `viewport-fit=cover` and then using the `env(safe-area-inset-*)` variables,

your app will automatically look perfect on every iPhone, from the mini SE to the massive Pro Max.

Would you like me to help you find where your main `index.html` or CSS file is located to make these changes?